

Trabajo Práctico — Unidad 8

Calidad – Análisis de Resultados y Acciones Correctivas

Alumno: Paddy Sussini
Fecha de entrega: 12 de junio de 2026

CASO 1 — Reserva duplicada en horario ocupado

Paso 1 — Interpretación del resultado

La evidencia muestra que un usuario (ID 182) logró reservar el mismo turno dos veces: mismo médico (Dra. Molina), misma fecha (12/11/2025) y mismo horario (10:00). El sistema respondió con éxito ambas veces y generó dos registros idénticos en la base de datos. El log confirma exactamente esto: dos POST /reservas con estado OK, el segundo incluso marcado como "duplicado" por el propio log pero aceptado de todas formas.

Estamos ante una **falla** observable (el sistema permite algo que no debería), originada por un **defecto** en el backend (ausencia de validación de unicidad en el endpoint de reservas). El **error** humano fue no incluir esa condición de control al implementar la funcionalidad de reserva. La secuencia completa es: error → defecto → falla, tal como lo define la teoría de la unidad.

Paso 2 — Clasificación del defecto

Tipo: **Error lógico**.

El problema no está en los datos ingresados (son válidos: médico, fecha y horario correctos) ni en la interfaz ni en la comunicación entre módulos. El error está en la lógica del backend: el sistema simplemente no pregunta si ya existe una reserva activa para esa combinación antes de confirmar la nueva. Es una falla estructural en el razonamiento del código — condición faltante en la función de reserva.

Paso 3 — Severidad y prioridad

Severidad: Alto.

La funcionalidad principal del sistema (reservar turnos) queda comprometida. Un médico puede terminar con dos pacientes citados para el mismo horario, lo que en un contexto real genera conflictos operativos graves. El sistema sigue funcionando parcialmente (se pueden hacer reservas), pero el flujo central está roto.

Prioridad: Alta.

Cualquier usuario puede reproducir este defecto en cualquier momento simplemente volviendo a hacer clic en confirmar, por lo que la frecuencia es alta. El costo de postergación también es alto: cada día que pasa se pueden acumular duplicados en la base que después hay que limpiar manualmente. Desde el punto de vista del negocio, no puede convivir con producción.

Paso 4 — Propuesta de acción correctiva

En el **backend**, antes de confirmar una nueva reserva, debe ejecutarse una consulta que verifique si ya existe un registro activo con la misma combinación de médico, fecha y horario. Si existe, el endpoint debe retornar HTTP 409 (Conflict) y no persistir el nuevo registro.

Complementariamente, conviene agregar una restricción UNIQUE en la tabla de reservas sobre los campos (medico_id, fecha, horario) como segunda capa de protección a nivel base de datos. El frontend debe mostrar un mensaje claro: "Ese horario ya no está disponible. Por favor elegí otro."

Criterio de aceptación verificable: al intentar reservar por segunda vez el mismo turno, el sistema debe rechazar la solicitud y mostrar el mensaje de error. No debe generarse ningún registro nuevo en la base.

Riesgos de regresión: la consulta previa puede impactar la performance si la tabla de reservas no tiene índices adecuados; también podría afectar flujos de cancelación y reasignación si comparten lógica con el módulo de creación.

Paso 5 — Análisis de Causa Raíz — técnica: 5 Whys

¿Por qué el sistema permitió la reserva duplicada?

Porque el endpoint POST /reservas no verifica si ya existe una reserva activa para ese médico, fecha y horario antes de confirmar.

¿Por qué no verifica?

Porque no hay ninguna condición de control implementada en la lógica del backend para ese caso.

¿Por qué no se implementó esa condición?

Porque no estaba especificada en los criterios de aceptación del requerimiento de reserva.

¿Por qué no estaba en los criterios de aceptación?

Porque el requerimiento solo decía "permitir reservar turno si está disponible" sin definir qué significa exactamente "disponible" ni cómo validarlo.

Causa raíz: Requerimiento incompleto. Los criterios de aceptación no definieron la condición de unicidad de reservas (mismo médico + fecha + horario = un solo turno activo). Al no estar definido, el desarrollador implementó la confirmación sin el control necesario.

Paso 6 — Plan de Re-testing

Caso a ejecutar: Reserva duplicada en mismo médico/fecha/horario.

Datos: Usuario: ID 182 | Médico: Dra. Molina | Fecha: 12/11/2025 | Horario: 10:00.

Pasos: (1) Iniciar sesión → (2) Seleccionar Dra. Molina → (3) Seleccionar 12/11/2025 → (4) Reservar horario 10:00 → confirmar → (5) Repetir los pasos 2 al 4 sin modificar ningún dato.

Resultado esperado correcto: La primera reserva se confirma normalmente. En el segundo intento, el sistema muestra el mensaje "Ese horario ya no está disponible" y no genera ningún registro nuevo en la base. El log no debe mostrar un segundo POST exitoso.

Paso 7 — Plan de Regresión

- 1. Reserva en distintos horarios del mismo médico:** verificar que el flujo estándar sigue funcionando y que la nueva validación no bloquea reservas legítimas.
- 2. Reservas de distintos usuarios en el mismo día:** confirmar que dos pacientes distintos pueden reservar horarios diferentes con la misma médica sin inconvenientes.
- 3. Cancelación y reasignación:** verificar que al cancelar una reserva, el horario queda disponible para una nueva reserva (la restricción UNIQUE no debe bloquear esto).
- 4. Listado de disponibilidad:** confirmar que los horarios ocupados aparecen como no disponibles en la pantalla de selección de turno.
- 5. Notificaciones de confirmación:** asegurarse de que el email/notificación de confirmación sigue enviándose correctamente tras una reserva exitosa.

Paso 8 — Ticket de Incidencia

ID	TP8-01
Título	Reserva duplicada permitida en horario ocupado
Resultado esperado	El sistema rechaza la segunda reserva y muestra: 'Horario no disponible. Por favor seleccioná otro.'
Resultado obtenido	El sistema confirma la segunda reserva con 'Tu turno ha sido reservado con éxito' y genera un duplicado en la base de datos.
Estado	Fail
Evidencia	Log del sistema: [INFO] POST /reservas - usuario 182 - OK / [INFO] POST /reservas - usuario 182 - OK (duplicado)
Tipo de defecto	Error lógico
Severidad	Alto
Prioridad	Alta
Propuesta de corrección	Implementar validación en el backend que consulte si ya existe una reserva activa para (medico_id, fecha, horario) antes de confirmar. Retornar HTTP 409 si hay conflicto. Agregar restricción UNIQUE en la tabla de reservas.
Causa raíz	Requerimiento incompleto: los criterios de aceptación no definieron la condición de unicidad de reservas.
Plan de re-testing	Ejecutar doble reserva con Dra. Molina, 12/11/2025, 10:00. Segunda reserva debe ser rechazada con mensaje de error visible y sin nuevo registro en base.
Plan de regresión	Re-ejecutar pruebas de: reserva en distintos horarios, distintos usuarios, cancelación/reasignación, listado de disponibilidad, y notificaciones.
Fecha de registro	12/06/2026

CASO 2 — Formulario de registro acepta email inválido

Paso 1 — Interpretación del resultado

La evidencia muestra que al ingresar "laura.perez@@gmail..com" (email con doble @ y doble punto, claramente inválido) el sistema crea el usuario sin mostrar ningún mensaje de error. El dato queda persistido tal cual en la base. Luego, al intentar iniciar sesión, el sistema trata de enviar un email de verificación a esa dirección y el envío falla silenciosamente (error SMTP), dejando al usuario sin posibilidad de acceder.

El log confirma todo: "[WARN] Email validation skipped for field 'email'" indica que la validación existe pero fue omitida. El "[ERROR] SMTP - Invalid address" muestra la consecuencia concreta.

Estamos ante una **falla** doble: el usuario queda registrado con datos inválidos (no puede ingresar al sistema) y la base de datos queda contaminada con un email malformado. El **defecto** es la ausencia de validación en el campo email. El **error** fue deshabilitar o no implementar esa validación durante el desarrollo.

Paso 2 — Clasificación del defecto

Tipo: **Error de validación.**

El sistema no controla que el dato ingresado en el campo email cumpla un formato estándar antes de procesarlo. El log es explícito al respecto: la validación fue deliberadamente saltada ("skipped"). Esta es la definición exacta de un error de validación según la clasificación de la unidad: datos no controlados o ingresados en formato incorrecto que el sistema acepta sin restricción.

Paso 3 — Severidad y prioridad

Severidad: Alto.

El defecto bloquea completamente la funcionalidad del usuario registrado: no puede verificar su cuenta ni iniciar sesión. Es un bloqueo funcional total para ese usuario, aunque el sistema en general sigue operativo. La base de datos también queda con registros inválidos que pueden generar problemas en otros procesos (envío masivo de emails, auditorías, etc.).

Prioridad: Alta.

Cualquier usuario que cometa un error tipográfico al registrarse (algo muy frecuente) queda atrapado sin solución desde el sistema. La frecuencia de ocurrencia es alta. El costo de postergación también: se acumulan usuarios bloqueados y registros inválidos en la base. Además, el log evidencia que la validación fue activamente desactivada, lo que sugiere que puede haber otros campos sin validar.

Paso 4 — Propuesta de acción correctiva

En el **frontend**: agregar validación del campo email con una expresión regular estándar (RFC 5322) antes de habilitar el botón "Crear cuenta". El mensaje de error debe ser visible y claro: "El email ingresado no tiene un formato válido. Revisá que no tenga caracteres duplicados ni espacios."

En el **backend**: implementar la misma validación de formato como capa de seguridad independiente del frontend. El flag que desactiva la validación (evidenciado por el "skipped" del log) debe corregirse o eliminarse. El endpoint de registro no debe persistir ningún usuario con email de formato inválido.

Criterio de aceptación verificable: al ingresar "laura.perez@@gmail..com" y hacer clic en "Crear cuenta", el formulario muestra el error de validación y no se crea ningún registro en la base de datos.

Riesgos de regresión: la validación podría rechazar emails válidos pero poco comunes (ej: con subdominio, con guiones). Hay que asegurarse de que el regex usado no sea demasiado restrictivo.

Paso 5 — Análisis de Causa Raíz — técnica: 5 Whys

¿Por qué el sistema aceptó un email con formato inválido?

Porque el campo email no tiene validación de formato activa al momento del registro.

¿Por qué no tiene validación activa?

Porque el log dice explícitamente que fue omitida: "Email validation skipped for field 'email'".

¿Por qué fue omitida?

Posiblemente porque se decidió postergarlo para una etapa posterior o se priorizó velocidad de desarrollo.

¿Por qué se pudo postergar sin consecuencias visibles en testing?

Porque no se incluyó ningún caso de prueba con email de formato inválido en el plan de testing del módulo de registro.

Causa raíz: La validación del formato de email no fue incluida como criterio de aceptación del requerimiento de registro de usuarios, lo que habilitó que fuera omitida en el desarrollo y no cubierta en el testing.

Paso 6 — Plan de Re-testing

Caso a ejecutar: Registro de usuario con email de formato inválido.

Datos: Nombre: Laura | Apellido: Pérez | Email: laura.perez@@gmail..com | Contraseña: Clave123.

Pasos: (1) Ingresar los datos en el formulario de registro → (2) Hacer clic en "Crear cuenta".

Resultado esperado correcto: El formulario muestra un mensaje de error de validación ("El formato del email es inválido") y no crea ningún registro en la base de datos. El botón puede inhabilitarse hasta que el email tenga formato correcto.

Caso adicional: Registrar con email válido (laura.perez@gmail.com) → debe funcionar normalmente, para confirmar que la corrección no rompió el flujo exitoso.

Paso 7 — Plan de Regresión

1. **Registro exitoso con email válido:** verificar que el flujo completo de registro con datos correctos sigue funcionando sin interrupciones.

2. **Login con usuario existente:** confirmar que los usuarios ya registrados con email válido pueden iniciar sesión normalmente.

3. **Envío de email de verificación:** verificar que tras un registro exitoso el sistema envía correctamente el correo de verificación.

4. **Otros campos del formulario:** asegurarse de que el cambio no afectó las validaciones de nombre, apellido y contraseña.

5. **Recuperación de contraseña:** verificar que el campo email en el formulario de recuperación también aplica la validación de formato correctamente.

Paso 8 — Ticket de Incidencia

ID	TP8-02
Título	Formulario de registro acepta email con formato inválido sin mostrar error
Resultado esperado	El sistema rechaza el email 'laura.perez@@gmail..com' y muestra un mensaje de error de validación. No crea el usuario.
Resultado obtenido	El sistema acepta el email inválido, crea el usuario en la base, y falla silenciosamente al intentar enviar el email de verificación.
Estado	Fail
Evidencia	[WARN] Email validation skipped for field 'email' / [ERROR] SMTP - Invalid address
Tipo de defecto	Error de validación
Severidad	Alto
Prioridad	Alta
Propuesta de corrección	Implementar validación de formato RFC 5322 en frontend (con mensaje de error visible) y en backend (sin posibilidad de omitir). Corregir el mecanismo que permite saltar la validación del campo email.
Causa raíz	La validación del formato de email no fue definida como criterio de aceptación del requerimiento, lo que permitió que fuera omitida en el desarrollo sin ser detectada en testing.
Plan de re-testing	Ejecutar registro con email 'laura.perez@@gmail..com'. Resultado esperado: mensaje de error visible, sin creación de usuario en base.
Plan de regresión	Re-ejecutar pruebas de: registro exitoso, login, envío de verificación, otros campos del formulario, y recuperación de contraseña.
Fecha de registro	12/06/2026

CASO 3 — Certificados PDF con acentos mal impresos

Paso 1 — Interpretación del resultado

Al generar los certificados en PDF, los caracteres con acento y la ñe se muestran como secuencias de bytes ilegibles: "Maria GonzÃ¡lez" en lugar de "María González", y "RepÃºblica Argentina" en lugar de "República Argentina". El patrón es consistente con un problema de encoding: el sistema genera el texto en UTF-8 pero la fuente o el motor de renderizado lo interpreta como Latin-1 (ISO-8859-1), produciendo esa corrupción característica (Ã¡ = á, Ã¼ = ü, etc.).

El log confirma exactamente esto: "[WARN] Fallback de fuente por incompatibilidad UTF-8". El flujo funciona: el PDF se genera y se descarga sin errores visibles para el usuario, pero el contenido está mal.

Estamos ante una **falla** observable (texto ilegible en el certificado). El **defecto** está en la configuración del módulo generador de PDFs (fuente o encoding incorrecto). El **error** fue no configurar ni probar el soporte de caracteres del español al implementar el generador.

Paso 2 — Clasificación del defecto

Tipo: **Error de integración.**

El problema no está en la lógica del negocio (el sistema sabe qué texto debe mostrar) ni en la validación de datos ni en la interfaz de usuario. El problema está en cómo el módulo de negocio se integra con la librería generadora de PDFs: la comunicación entre ambos componentes no está configurada correctamente en cuanto al encoding de caracteres. El fallback silencioso de fuente que registra el log es una señal directa de falla en esa integración.

Paso 3 — Severidad y prioridad

Severidad: Medio.

Técnicamente, el flujo no se interrumpe: el PDF se genera, se puede descargar e imprimir. La información no se pierde, solo se muestra incorrectamente. No es un bloqueo funcional en el sentido estricto. Sin embargo, el contenido del documento está corrupto, lo que compromete su validez.

Prioridad: Alta.

Aquí aplica exactamente el mini-ejemplo del material de la unidad sobre severidad y prioridad: severidad media con prioridad alta por contexto de negocio. El sistema va a usarse la semana próxima para generar más de 500 certificados oficiales que se imprimirán y entregarán en un acto institucional. Reimprimir 500 certificados tiene un costo económico y reputacional enorme. El comentario del responsable de sistema ("no es grave, después se corrige") ignora completamente el contexto: en este caso, el contexto lo vuelve urgente.

Paso 4 — Propuesta de acción correctiva

En el **módulo generador de PDFs**: reemplazar la fuente actualmente configurada por una que soporte completamente UTF-8 y caracteres latinos extendidos (ej: DejaVu Sans, o cualquier fuente TTF embebida que incluya el rango de caracteres necesario). Si se usa una librería como ReportLab (Python), basta con registrar y usar una fuente TTF compatible.

Adicionalmente, verificar que la cadena de texto se pasa al generador codificada en UTF-8 en todos los puntos del flujo, sin conversiones intermedias a Latin-1.

Criterio de aceptación verificable: al generar el certificado de "María González", el PDF debe mostrar exactamente "María González" y "República Argentina", sin ningún carácter extraño. Verificar también con nombres que incluyan é, í, ó, ú y ñ.

Riesgos de regresión: el cambio de fuente puede alterar el layout visual del certificado (márgenes, espaciado, tamaño percibido del texto). Hay que verificar que el diseño del certificado se mantiene correcto tras el cambio.

Paso 5 — Análisis de Causa Raíz — técnica: Diagrama de Ishikawa

Organizando las posibles causas por categoría:

Implementación/código: La fuente configurada en el módulo generador de PDFs no incluye soporte para caracteres UTF-8 con acentos del español. El sistema hace un "fallback" silencioso a otra fuente sin notificarlo al usuario ni detener el proceso.

Requerimientos: No se especificó como requisito que el sistema debe soportar caracteres especiales del español (á, é, í, ó, ú, ñ) en los documentos generados.

Proceso / Testing: No se realizaron pruebas de generación de certificados con nombres que contengan acentos antes de habilitarlo para producción. Si se hubiera testado con "María" o "González", el problema hubiera aparecido de inmediato.

Causa raíz principal: La fuente configurada en el módulo de generación de PDFs no soporta el conjunto de caracteres UTF-8 necesario para el español, y esta incompatibilidad no fue detectada porque no hubo casos de prueba con datos que incluyeran acentos o ñe.

Paso 6 — Plan de Re-testing

Caso a ejecutar: Generación de certificado para estudiante con nombre acentuado.

Datos: Estudiante: María González | Carrera: [nombre de carrera] | Texto fijo: incluye "República Argentina".

Pasos: (1) Ingresar al sistema → (2) Seleccionar la estudiante María González → (3) Generar y descargar el certificado en PDF → (4) Abrir el PDF y verificar el texto.

Resultado esperado correcto: El PDF muestra "María González" y "República Argentina" correctamente, sin ningún carácter corrupto.

Casos adicionales: repetir con nombres que incluyan é, í, ó, ú y ñ para confirmar cobertura completa del rango de caracteres.

Paso 7 — Plan de Regresión

1. **Layout visual del certificado:** verificar que el diseño, márgenes y formato del certificado se mantienen correctos tras el cambio de fuente.

2. **Nombres sin caracteres especiales:** confirmar que los certificados de estudiantes sin acentos en su nombre siguen generándose correctamente.

3. **Descarga del archivo:** verificar que el PDF se genera y descarga sin errores de servidor ni timeouts.

4. **Generación masiva:** hacer una prueba de generación de varios certificados seguidos para confirmar que la nueva fuente no genera problemas de memoria o degradación de performance.

5. **Otros documentos del sistema:** verificar si existen otros módulos que generen PDFs (recibos, constancias, etc.) y confirmar que también muestran correctamente los caracteres especiales.

Paso 8 — Ticket de Incidencia

ID	TP8-03
Título	Certificados PDF con caracteres acentuados mal codificados (UTF-8 / fuente incompatible)
Resultado esperado	El PDF muestra correctamente 'María González' y 'República Argentina' sin caracteres corruptos.
Resultado obtenido	El PDF muestra 'Maria GonzÃ¡lez' y 'RepÃºblica Argentina'. El problema afecta todos los caracteres con acento (á, é, í, ó, ú, ñ).
Estado	Fail
Evidencia	[INFO] Generando PDF certificado_id=582 / [WARN] Fallback de fuente por incompatibilidad UTF-8
Tipo de defecto	Error de integración
Severidad	Medio
Prioridad	Alta (contexto: 500+ certificados oficiales para acto institucional la próxima semana)
Propuesta de corrección	Reemplazar la fuente del generador de PDFs por una con soporte UTF-8 completo (ej: DejaVu TTF embebida). Verificar que el texto llega codificado en UTF-8 a lo largo de todo el flujo.
Causa raíz	La fuente configurada no soporta caracteres UTF-8 del español. El sistema hace fallback silencioso. No hubo casos de prueba con nombres acentuados.
Plan de re-testing	Generar certificado de 'María González'. El PDF debe mostrar el nombre y 'República Argentina' sin caracteres corruptos.
Plan de regresión	Verificar: layout visual del certificado, nombres sin acento, descarga correcta, generación masiva, y otros documentos PDF del sistema.
Fecha de registro	12/06/2026

CASO 4 — Carrito de compras recalcula mal el total con descuento

Paso 1 — Interpretación del resultado

El sistema aplica un descuento del 20% cuando debería aplicar el 10%. La matemática es clara: con subtotal \$30.000 y 10% de descuento, el resultado correcto es \$27.000 (\$30.000 - \$3.000). Sin embargo el sistema muestra \$24.000, que corresponde a un descuento de \$6.000, es decir el 20%. El log lo confirma: el sistema recibe correctamente `discount_percent=0.1`, el subtotal es 30.000, pero el total calculado es 24.000. La función de cálculo está duplicando el descuento de alguna forma.

La UI muestra "Descuento aplicado: 10%" (correcto) pero el número final está mal. La inconsistencia entre lo que dice la etiqueta y lo que muestra el número es en sí misma otra señal de que el error está en la lógica de cálculo, no en la presentación.

Este es un caso típico de **regresión**: el defecto fue introducido por una modificación reciente en una función de cálculo compartida. La **falla** es el total incorrecto. El **defecto** está en la función de descuento. El **error** fue del desarrollador que modificó la función sin considerar los módulos que dependen de ella.

Paso 2 — Clasificación del defecto

Tipo: **Error lógico**.

El problema está directamente en el cálculo matemático dentro de la función de descuentos: recibe el porcentaje correcto (0.1) pero devuelve un resultado que equivale al doble (0.2). Puede ser que el descuento se aplique dos veces, que haya un factor mal calculado, o que una modificación reciente haya introducido una operación duplicada. En todos los casos, es una falla en la estructura lógica del código, no en la validación de datos ni en la interfaz ni en la comunicación entre módulos.

Paso 3 — Severidad y prioridad

Severidad: Crítico.

El error afecta directamente la integridad de los datos financieros: los clientes pagan menos de lo que corresponde, lo que genera pérdidas económicas reales para el negocio, inconsistencias contables (el total registrado en la base no corresponde al precio real) y posibles problemas de auditoría. No hay solución alternativa viable mientras el bug esté activo: cualquier compra con código de descuento resulta en un cobro incorrecto.

Prioridad: Alta.

La frecuencia es alta (cualquier compra con código de descuento activa el bug) y el impacto económico es directo e inmediato. Cada transacción completada mientras el bug está activo genera una pérdida concreta. El costo de postergación es máximo. Además, el defecto es una regresión introducida recientemente, lo que sugiere que puede haber otros módulos afectados por el mismo cambio.

Paso 4 — Propuesta de acción correctiva

En el **backend**: revisar la función de cálculo de descuentos identificando el punto donde el porcentaje se duplica. El log muestra que el input es correcto (`subtotal=30000`, `discount_percent=0.1`) pero el output no (`total=24000` en vez de `27000`), por lo que el error está dentro de la función. El desarrollador debe revisar el commit de la semana pasada que modificó la función de cálculo para otro módulo y encontrar qué cambio introdujo la duplicación.

La fórmula correcta es: $\text{total} = \text{subtotal} * (1 - \text{discount_percent})$, o equivalentemente: $\text{total} = \text{subtotal} - (\text{subtotal} * \text{discount_percent})$. Si alguna de las dos formas quedó sumada o anidada con otra aplicación del descuento, ahí está el bug.

Criterio de aceptación verificable: con subtotal \$30.000 y código DESCUENTO10 (10%), el total mostrado y registrado en la base debe ser exactamente \$27.000.

Riesgos de regresión: si la función de cálculo es compartida entre módulos (carrito, facturación, reportes de ventas), la corrección puede tener impacto amplio y hay que hacer regresión sobre todos esos módulos.

Paso 5 — Análisis de Causa Raíz — técnica: 5 Whys

¿Por qué el total calculado es \$24.000 en lugar de \$27.000?

Porque la función de cálculo de descuentos está aplicando el 20% en lugar del 10%.

¿Por qué está aplicando el 20%?

Porque la función está duplicando el descuento de alguna forma (lo aplica dos veces o usa un factor incorrecto).

¿Por qué está duplicando el descuento?

Porque la semana pasada se modificó la función de cálculo para otro módulo y ese cambio introdujo un error en la lógica de descuentos del carrito.

¿Por qué el cambio no fue detectado antes del deploy?

Porque no se ejecutaron pruebas de regresión sobre el módulo de descuentos del carrito después de modificar la función compartida.

Causa raíz: La modificación de una función de cálculo compartida entre módulos se realizó sin ejecutar un plan de regresión sobre los módulos dependientes, lo que permitió que el error llegara a producción sin ser detectado.

Paso 6 — Plan de Re-testing

Caso a ejecutar: Compra con tres productos y código de descuento del 10%.

Datos: Producto A: \$10.000 | Producto B: \$8.000 | Producto C: \$12.000 | Subtotal: \$30.000 | Código: DESCUENTO10 (10%).

Pasos: (1) Iniciar sesión → (2) Agregar los tres productos al carrito → (3) Aplicar el código DESCUENTO10 → (4) Verificar los valores mostrados → (5) Confirmar la compra → (6) Verificar el total registrado en la base de datos.

Resultado esperado correcto: Subtotal: \$30.000 | Descuento: 10% (\$3.000) | Total: \$27.000. El valor registrado en la base de datos (total_final) debe ser 27000, no 24000.

Caso adicional: probar con otro porcentaje (ej: 20%) para validar que el cálculo es correcto en todos los casos y no solo para el 10%.

Paso 7 — Plan de Regresión

1. **Módulo de precios donde se hizo el cambio la semana pasada:** verificar que ese módulo sigue calculando correctamente y que la corrección del bug del carrito no rompe lo que se había arreglado antes.

2. **Compra sin código de descuento:** confirmar que el total es igual al subtotal cuando no hay descuento aplicado.

3. **Distintos porcentajes de descuento:** probar con códigos de 5%, 15% y 20% para validar que la función de cálculo es correcta en todos los casos.
4. **Conciliación de totales en base de datos:** verificar que el campo `total_final` registrado en la base coincide exactamente con lo mostrado en pantalla al usuario.
5. **Proceso de facturación:** verificar que los comprobantes y facturas generados reflejan el total correcto y que el proceso de cierre contable no se ve afectado.

Paso 8 — Ticket de Incidencia

ID	TP8-04
Título	Carrito aplica descuento del 20% cuando el código indica 10% (regresión en función de cálculo)
Resultado esperado	Con subtotal \$30.000 y código DESCUENTO10, el total debe ser \$27.000 (10% de descuento = \$3.000).
Resultado obtenido	El sistema muestra y registra total \$24.000, equivalente a un descuento del 20% (\$6.000). La base guarda <code>total_final</code> : 24000 con <code>descuento_aplicado</code> : 0.10.
Estado	Fail
Evidencia	[DEBUG] subtotal=30000 / [DEBUG] discount_percent=0.1 / [DEBUG] total=24000
Tipo de defecto	Error lógico
Severidad	Crítico
Prioridad	Alta
Propuesta de corrección	Revisar y corregir la función de cálculo de descuentos. Analizar el commit de la semana pasada que modificó la función compartida. La fórmula correcta es: $\text{total} = \text{subtotal} * (1 - \text{discount_percent})$. Ejecutar suite de regresión completa sobre el módulo de carrito y facturación.
Causa raíz	Se modificó una función de cálculo compartida entre módulos sin ejecutar pruebas de regresión sobre los módulos dependientes, permitiendo que el error llegara a producción.
Plan de re-testing	Ejecutar el flujo completo: tres productos + código DESCUENTO10. Resultado esperado: total \$27.000 en pantalla y en base de datos.
Plan de regresión	Re-ejecutar pruebas de: módulo de precios modificado, compra sin descuento, distintos porcentajes de descuento, conciliación de totales en base, y proceso de facturación.
Fecha de registro	12/06/2026